# Data Engineering Programming

## Trabalho Final - FIAP MBA - Prof. Marcelo Barbosa Pinto

Relatório de pedidos de 2025 com pagamento recusado (status=false) e fraude=false, em PySpark, orientado a objetos e com injeção de dependências.


| Integrante | RM |
|---|---|
| Priscila Cristina Mendes Cardoso | 373153 |
| Andre Victor Bagatin | 374773 |
| Felipe Nogueira da Silva | 373111 |
| Heloisa Cordeiro Silva | 373370 |
| Leonardo Machado de Andrade | 373150 |
| Nicolas Galhardo Lhen | 374205 |


**Repositório público no GitHub:**

`https://github.com/AndreBagatin/fiap-dep-pyspark-relatorio-pedidos`

## Arquitetura e injeção de dependências

**main.py** é o Aggregation Root: a classe Application instancia Settings, SparkSessionManager, DataReader, RelatorioPedidos, ParquetWriter e Pipeline, injetando-os via construtor. Nenhuma outra classe cria suas dependências.

Pacotes: **config** (Settings), **session** (SparkSessionManager), **io_utils** (schemas, DataReader, ParquetWriter), **business** (RelatorioPedidos) e **pipeline** (Pipeline: leitura, negócio e escrita).

Todos os DataFrames usam StructType explícito (inclusive o struct aninhado avaliacao_fraude); não há inferência de schema nem notebooks.

## Decisões de negócio

**Valor total do pedido** = valor_unitario x quantidade (pedidos). O valor_pagamento difere em 21.297 dos 60.096 pedidos (ex.: desconto no PIX), portanto não representa o valor do pedido.

**Join 1:1**: 60.096 pedidos e 60.096 pagamentos, ambos com id_pedido único; inner join por id_pedido, sem duplicidade.

**Filtros**: ano de data_criacao = 2025; status = false e avaliacao_fraude.fraude = false. Resultado real: 1.473 pedidos.

**Ordenação**: uf, forma_pagamento e data do pedido; gravação em parquet (1 partição para preservar a ordem).

## Logging e tratamento de erros

RelatorioPedidos importa logging, configura logging.basicConfig(level=INFO) e registra a contagem de cada etapa. Falhas caem em try/except, são registradas com logging.error e relançadas como RelatorioPedidosError.

## Como executar (detalhes no README.md do repositório)

pip install -r requirements.txt; pip install .; python scripts/download_datasets.py; python main.py; pytest

Repositório: https://github.com/AndreBagatin/fiap-dep-pyspark-relatorio-pedidos

## Aggregation Root: instancia e injeta todas as dependências

main.py - linhas 1 a 17

```python
"""Aggregation Root: único lugar que instancia as dependências e as injeta."""

import logging
import sys

from relatorio_pedidos.business.relatorio_pedidos import RelatorioPedidos
from relatorio_pedidos.config.settings import Settings
from relatorio_pedidos.io_utils.readers import DataReader
from relatorio_pedidos.io_utils.writers import ParquetWriter
from relatorio_pedidos.pipeline.pipeline import Pipeline
from relatorio_pedidos.session.spark_session_manager import SparkSessionManager

logger = logging.getLogger(__name__)


class Application:
    """Monta o grafo de dependências e executa o pipeline."""
```

main.py - linhas 19 a 38

```
19      def __init__(self, config_path: str | None = None):
20          self._config_path = config_path
21
22      def run(self) -> int:
23          settings = Settings.load(self._config_path)
24          session_manager = SparkSessionManager(settings)
25          try:
26              spark = session_manager.get_session()
27              pipeline = Pipeline(
28                  reader=DataReader(spark, settings),
29                  relatorio=RelatorioPedidos(ano=settings.ano_relatorio),
30                  writer=ParquetWriter(settings),
31              )
32              pipeline.run()
33              return 0
34          except Exception:
35              logger.exception("Execução interrompida por erro")
36              return 1
37          finally:
38              session_manager.stop()
```

main.py - linhas 41 a 42

```
41  if __name__ == "__main__":
42      sys.exit(Application(config_path=sys.argv[1] if len(sys.argv) > 1 else None).run())
```

## Configurações externas (caminhos, opções de leitura, ano)

config/settings.yaml - linhas 1 a 12

```yaml
# Configurações do pipeline (ver src/relatorio_pedidos/config/settings.py).
# Variáveis de ambiente PEDIDOS_PATH, PAGAMENTOS_PATH, OUTPUT_PATH e ANO_RELATORIO
# têm precedência sobre este arquivo.
spark:
  app_name: relatorio-pedidos-recusados-legitimos
  master: "local[*]"
  shuffle_partitions: 8

paths:
  pedidos: data/input/datasets-csv-pedidos/data/pedidos/
  pagamentos: data/input/dataset-json-pagamentos/data/pagamentos/
  output: data/output/relatorio_pedidos
```

config/settings.yaml - linhas 14 a 28

```yaml
file_options:
  mode: FAILFAST
  pedidos_csv:
    sep: ";"
    header: true
    compression: gzip
  pagamentos_json:
    compression: gzip

output:
  mode: overwrite
  partitions: 1

negocio:
  ano_relatorio: 2025
```

## Pacote config: classe Settings

src/relatorio_pedidos/config/settings.py - linhas 1 a 19

```python
"""Configurações centralizadas do projeto.

Ordem de precedência (da menor para a maior):
1. valores padrão definidos na classe ``Settings``;
2. arquivo YAML (``config/settings.yaml`` ou o indicado em ``RELATORIO_CONFIG``);
3. variáveis de ambiente (``PEDIDOS_PATH``, ``PAGAMENTOS_PATH``, ``OUTPUT_PATH``,
   ``ANO_RELATORIO``).
"""

import os
from dataclasses import dataclass, replace
from pathlib import Path

import yaml


@dataclass(frozen=True)
class Settings:
    """Parâmetros de execução do pipeline."""
```

src/relatorio_pedidos/config/settings.py - linhas 21 a 35

```
21      # Spark
22      app_name: str = "relatorio-pedidos-recusados-legitimos"
23      master: str = "local[*]"
24      shuffle_partitions: int = 8
25
26      # Entrada
27      pedidos_path: str = "data/input/datasets-csv-pedidos/data/pedidos/"
28      pagamentos_path: str = "data/input/dataset-json-pagamentos/data/pagamentos/"
29
30      # Opções de leitura
31      pedidos_sep: str = ";"
32      pedidos_header: bool = True
33      pedidos_compression: str = "gzip"
34      pagamentos_compression: str = "gzip"
35      read_mode: str = "FAILFAST"
```

src/relatorio_pedidos/config/settings.py - linhas 37 a 46

```
37      # Saída
38      output_path: str = "data/output/relatorio_pedidos"
39      output_mode: str = "overwrite"
40      output_partitions: int = 1
41
42      # Regras de negócio
43      ano_relatorio: int = 2025
44
45      ENV_CONFIG = "RELATORIO_CONFIG"
46      DEFAULT_CONFIG_FILE = "config/settings.yaml"
```

src/relatorio_pedidos/config/settings.py - linhas 48 a 56

```python
    @classmethod
    def load(cls, path: str | None = None) -> "Settings":
        """Carrega as configurações combinando padrões, YAML e variáveis de ambiente."""
        settings = cls()
        yaml_path = Path(path or os.environ.get(cls.ENV_CONFIG) or cls.DEFAULT_CONFIG_FILE)
        if yaml_path.is_file():
            with open(yaml_path, "r", encoding="utf-8") as file:
                settings = cls._from_dict(settings, yaml.safe_load(file) or {})
        return cls._from_env(settings)
```

src/relatorio_pedidos/config/settings.py - linhas 58 a 77

```python
    @staticmethod
    def _from_dict(base: "Settings", raw: dict) -> "Settings":
        spark = raw.get("spark", {})
        paths = raw.get("paths", {})
        pedidos_csv = raw.get("file_options", {}).get("pedidos_csv", {})
        pagamentos_json = raw.get("file_options", {}).get("pagamentos_json", {})
        output = raw.get("output", {})
        negocio = raw.get("negocio", {})
        valores = {
            "app_name": spark.get("app_name"),
            "master": spark.get("master"),
            "shuffle_partitions": spark.get("shuffle_partitions"),
            "pedidos_path": paths.get("pedidos"),
            "pagamentos_path": paths.get("pagamentos"),
            "pedidos_sep": pedidos_csv.get("sep"),
            "pedidos_header": pedidos_csv.get("header"),
            "pedidos_compression": pedidos_csv.get("compression"),
            "pagamentos_compression": pagamentos_json.get("compression"),
            "read_mode": raw.get("file_options", {}).get("mode"),
            "output_path": paths.get("output"),
```

src/relatorio_pedidos/config/settings.py - linhas 78 a 93

```
            "output_mode": output.get("mode"),
            "output_partitions": output.get("partitions"),
            "ano_relatorio": negocio.get("ano_relatorio"),
        }
        return replace(base, **{k: v for k, v in valores.items() if v is not None})

    @staticmethod
    def _from_env(base: "Settings") -> "Settings":
        env = os.environ
        valores = {
            "pedidos_path": env.get("PEDIDOS_PATH"),
            "pagamentos_path": env.get("PAGAMENTOS_PATH"),
            "output_path": env.get("OUTPUT_PATH"),
            "ano_relatorio": int(env["ANO_RELATORIO"]) if env.get("ANO_RELATORIO") else None,
        }
        return replace(base, **{k: v for k, v in valores.items() if v is not None})
```

## Pacote session: SparkSessionManager

src/relatorio_pedidos/session/spark_session_manager.py - linhas 1 a 11

```
from pyspark.sql import SparkSession

from relatorio_pedidos.config.settings import Settings


class SparkSessionManager:
    """Gerencia o ciclo de vida (criar, obter e encerrar) da SparkSession."""

    def __init__(self, settings: Settings):
        self._settings = settings
        self._spark: SparkSession | None = None
```

src/relatorio_pedidos/session/spark_session_manager.py - linhas 13 a 30

```
13      def get_session(self) -> SparkSession:
14          """Cria a sessão na primeira chamada e a reutiliza nas seguintes."""
15          if self._spark is None:
16              self._spark = (
17                  SparkSession.builder.appName(self._settings.app_name)
18                  .master(self._settings.master)
19                  .config("spark.sql.shuffle.partitions", self._settings.shuffle_partitions)
20                  .config("spark.ui.showConsoleProgress", "false")
21                  .getOrCreate()
22              )
23              self._spark.sparkContext.setLogLevel("WARN")
24          return self._spark
25
26      def stop(self) -> None:
27          """Encerra a sessão, se estiver ativa."""
28          if self._spark is not None:
29              self._spark.stop()
30              self._spark = None
```

## Schemas explícitos (StructType), inclusive aninhado

src/relatorio_pedidos/io_utils/schemas.py - linhas 1 a 13

```python
from pyspark.sql.types import (
    BooleanType,
    DoubleType,
    LongType,
    StringType,
    StructField,
    StructType,
    TimestampType,
)


class PedidosSchema:
    """Schema explícito do dataset de pedidos (CSV)."""
```

src/relatorio_pedidos/io_utils/schemas.py - linhas 15 a 31

```
15      @staticmethod
16      def get() -> StructType:
17          return StructType(
18              [
19                  StructField("id_pedido", StringType(), True),
20                  StructField("produto", StringType(), True),
21                  StructField("valor_unitario", DoubleType(), True),
22                  StructField("quantidade", LongType(), True),
23                  StructField("data_criacao", TimestampType(), True),
24                  StructField("uf", StringType(), True),
25                  StructField("id_cliente", LongType(), True),
26              ]
27          )
28
29
30  class PagamentosSchema:
31      """Schema explícito do dataset de pagamentos (JSON), incluindo o objeto aninhado."""
```

src/relatorio_pedidos/io_utils/schemas.py - linhas 33 a 40

```
33      @staticmethod
34      def avaliacao_fraude() -> StructType:
35          return StructType(
36              [
37                  StructField("fraude", BooleanType(), True),
38                  StructField("score", DoubleType(), True),
39              ]
40          )
```

src/relatorio_pedidos/io_utils/schemas.py - linhas 42 a 53

```
42      @staticmethod
43      def get() -> StructType:
44          return StructType(
45              [
46                  StructField("id_pedido", StringType(), True),
47                  StructField("forma_pagamento", StringType(), True),
48                  StructField("valor_pagamento", DoubleType(), True),
49                  StructField("status", BooleanType(), True),
50                  StructField("data_processamento", TimestampType(), True),
51                  StructField("avaliacao_fraude", PagamentosSchema.avaliacao_fraude(), True),
52              ]
53          )
```

## Pacote io_utils: leitura de CSV e JSON com schema

src/relatorio_pedidos/io_utils/readers.py - linhas 1 a 12

```
1  from pyspark.sql import DataFrame, SparkSession
2
3  from relatorio_pedidos.config.settings import Settings
4  from relatorio_pedidos.io_utils.schemas import PagamentosSchema, PedidosSchema
5
6
7  class DataReader:
8      """Lê os datasets de entrada sempre com schema explícito (sem inferência)."""
9
10     def __init__(self, spark: SparkSession, settings: Settings):
11         self._spark = spark
12         self._settings = settings
```

src/relatorio_pedidos/io_utils/readers.py - linhas 14 a 25

```
14      def read_pedidos(self) -> DataFrame:
15          """Lê os pedidos (CSV com separador e cabeçalho definidos na configuração)."""
16          return (
17              self._spark.read.schema(PedidosSchema.get())
18              .option("compression", self._settings.pedidos_compression)
19              .option("mode", self._settings.read_mode)
20              .csv(
21                  self._settings.pedidos_path,
22                  header=self._settings.pedidos_header,
23                  sep=self._settings.pedidos_sep,
24              )
25          )
```

src/relatorio_pedidos/io_utils/readers.py - linhas 27 a 34

```
27      def read_pagamentos(self) -> DataFrame:
28          """Lê os pagamentos (JSON, um objeto por linha, com avaliacao_fraude aninhada)."""
29          return (
30              self._spark.read.schema(PagamentosSchema.get())
31              .option("compression", self._settings.pagamentos_compression)
32              .option("mode", self._settings.read_mode)
33              .json(self._settings.pagamentos_path)
34          )
```

## Pacote io_utils: escrita em parquet

src/relatorio_pedidos/io_utils/writers.py - linhas 1 a 13

```python
from pyspark.sql import DataFrame

from relatorio_pedidos.config.settings import Settings


class ParquetWriter:
    """Grava o relatório em formato parquet."""

    def __init__(self, settings: Settings):
        self._settings = settings

    def write(self, df: DataFrame) -> str:
        """Grava o DataFrame e devolve o caminho de saída.
```

src/relatorio_pedidos/io_utils/writers.py - linhas 15 a 23

```python
        O ``coalesce`` é aplicado depois da ordenação e não a desfaz: ele apenas
        junta partições vizinhas, preservando a ordem global do relatório.
        """
        (
            df.coalesce(self._settings.output_partitions)
            .write.mode(self._settings.output_mode)
            .parquet(self._settings.output_path)
        )
        return self._settings.output_path
```

## Pacote business: regras, logging e try/except

src/relatorio_pedidos/business/relatorio_pedidos.py - linhas 1 a 16

```python
import logging
from typing import Callable

from pyspark.sql import DataFrame
from pyspark.sql import functions as F

logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s")
logger = logging.getLogger(__name__)


class RelatorioPedidosError(Exception):
    """Erro controlado lançado quando uma etapa da lógica de negócio falha."""


class RelatorioPedidos:
    """Regras de negócio do relatório de pedidos com pagamento recusado e fraude=false.
```

src/relatorio_pedidos/business/relatorio_pedidos.py - linhas 18 a 35

```
18      Etapas: filtro do ano -> valor total -> filtro de pagamentos -> join ->
19      seleção de colunas -> ordenação.
20      """
21
22      COLUNAS_RELATORIO = ["id_pedido", "uf", "forma_pagamento", "valor_total", "data_pedido"]
23
24      def __init__(self, ano: int):
25          self._ano = ano
26
27      def filtrar_pagamentos_recusados_legitimos(self, pagamentos: DataFrame) -> DataFrame:
28          """Mantém pagamentos recusados (status=false) avaliados como legítimos (fraude=false)."""
29          return pagamentos.filter(
30              (F.col("status") == F.lit(False)) & (F.col("avaliacao_fraude.fraude") == F.lit(False))
31          )
32
33      def filtrar_ano(self, pedidos: DataFrame) -> DataFrame:
34          """Mantém apenas pedidos criados no ano do relatório."""
35          return pedidos.filter(F.year(F.col("data_criacao")) == self._ano)
```

src/relatorio_pedidos/business/relatorio_pedidos.py - linhas 37 a 53

```
37      def calcular_valor_total(self, pedidos: DataFrame) -> DataFrame:
38          """Valor total do pedido = valor_unitario * quantidade."""
39          return pedidos.withColumn("valor_total", F.col("valor_unitario") * F.col("quantidade"))
40
41      def juntar(self, pedidos: DataFrame, pagamentos: DataFrame) -> DataFrame:
42          """Junção interna pedido x pagamento por id_pedido (relação 1:1 nos datasets)."""
43          return pedidos.join(pagamentos, on="id_pedido", how="inner")
44
45      def selecionar_colunas(self, df: DataFrame) -> DataFrame:
46          """Seleciona as 5 colunas do relatório."""
47          return df.select(
48              "id_pedido",
49              "uf",
50              "forma_pagamento",
51              "valor_total",
52              F.col("data_criacao").alias("data_pedido"),
53          )
```

src/relatorio_pedidos/business/relatorio_pedidos.py - linhas 55 a 73

```python
    def ordenar(self, df: DataFrame) -> DataFrame:
        """Ordena por UF, forma de pagamento e data do pedido (id_pedido desempata)."""
        return df.orderBy("uf", "forma_pagamento", "data_pedido", "id_pedido")

    def gerar_relatorio(self, pedidos: DataFrame, pagamentos: DataFrame) -> DataFrame:
        """Executa todas as etapas e devolve o DataFrame final do relatório."""
        logger.info("Iniciando a geração do relatório (ano %s)", self._ano)
        pedidos = self._executar("Filtro do ano", self.filtrar_ano, pedidos)
        pedidos = self._executar("Cálculo do valor total", self.calcular_valor_total, pedidos)
        pagamentos = self._executar(
            "Filtro de pagamentos recusados e legítimos",
            self.filtrar_pagamentos_recusados_legitimos,
            pagamentos,
        )
        df = self._executar("Join pedidos x pagamentos", self.juntar, pedidos, pagamentos)
        df = self._executar("Seleção de colunas", self.selecionar_colunas, df)
        df = self._executar("Ordenação", self.ordenar, df)
        logger.info("Relatório gerado com sucesso")
        return df
```

src/relatorio_pedidos/business/relatorio_pedidos.py - linhas 75 a 83

```python
    def _executar(self, etapa: str, funcao: Callable[..., DataFrame], *args: DataFrame) -> DataFrame:
        """Roda uma etapa registrando a contagem de registros; falhas viram RelatorioPedidosError."""
        try:
            resultado = funcao(*args)
            logger.info("Etapa '%s' concluída: %d registros", etapa, resultado.count())
            return resultado
        except Exception as erro:
            logger.error("Falha na etapa '%s': %s", etapa, erro)
            raise RelatorioPedidosError(f"Falha na etapa '{etapa}'") from erro
```

## Pacote pipeline: orquestração

src/relatorio_pedidos/pipeline/pipeline.py - linhas 1 a 15

```python
import logging

from relatorio_pedidos.business.relatorio_pedidos import RelatorioPedidos
from relatorio_pedidos.io_utils.readers import DataReader
from relatorio_pedidos.io_utils.writers import ParquetWriter

logger = logging.getLogger(__name__)


class Pipeline:
    """Orquestra leitura -> transformação -> escrita.

    Não cria suas dependências: todas chegam prontas pelo construtor,
    montadas no Aggregation Root (main.py).
    """
```

src/relatorio_pedidos/pipeline/pipeline.py - linhas 17 a 30

```python
    def __init__(self, reader: DataReader, relatorio: RelatorioPedidos, writer: ParquetWriter):
        self._reader = reader
        self._relatorio = relatorio
        self._writer = writer

    def run(self) -> str:
        """Executa o pipeline completo e devolve o caminho do parquet gerado."""
        logger.info("Pipeline iniciado")
        pedidos = self._reader.read_pedidos()
        pagamentos = self._reader.read_pagamentos()
        df = self._relatorio.gerar_relatorio(pedidos, pagamentos)
        caminho = self._writer.write(df)
        logger.info("Pipeline concluído. Parquet gravado em: %s", caminho)
        return caminho
```

## Empacotamento: pyproject.toml

pyproject.toml - linhas 1 a 20

```toml
[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "fiap-dep-pyspark-relatorio-pedidos"
version = "1.0.0"
description = "Relatório de pedidos com pagamento recusado e fraude=false (PySpark, POO e injeção de dependências)."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [
  { name = "Grupo FIAP - Data Engineering Programming" },
]
classifiers = [
    "Programming Language :: Python :: 3",
    "Operating System :: OS Independent",
]
dependencies = [
    "pyspark>=4.0.0,<5.0.0",
```

pyproject.toml - linhas 21 a 35

```
    "pyyaml>=6.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "ruff>=0.5",
    "build>=1.0",
    "pygments>=2.17",
    "pillow>=10.0",
    "reportlab>=4.0",
]

[tool.setuptools.packages.find]
where = ["src"]
```

pyproject.toml - linhas 37 a 50

```
[tool.pytest.ini_options]
pythonpath = ["src", "."]
testpaths = ["tests"]
addopts = "-v"

[tool.ruff]
line-length = 110
src = ["src", "tests", "."]

[tool.ruff.lint]
select = ["E", "F", "W", "I"]

[tool.ruff.lint.isort]
known-first-party = ["main", "relatorio_pedidos"]
```

## Empacotamento: requirements.txt

requirements.txt - linhas 1 a 9

```
pyspark==4.2.0
PyYAML==6.0.3
pytest==9.1.1
ruff==0.17.0
build==1.6.1
# Usados apenas por scripts/gerar_evidencias.py (PDF de entrega)
Pygments==2.21.0
pillow==12.3.0
reportlab==5.0.1
```

## Empacotamento: MANIFEST.in

MANIFEST.in - linhas 1 a 8

```
include README.md
include requirements.txt
include main.py
include config/settings.yaml
recursive-include scripts *.py
recursive-include tests *.py
recursive-include src *.py
global-exclude __pycache__ *.py[cod]
```

## Testes: fixtures com schema explícito

tests/conftest.py - linhas 1 a 11

```python
import os
import sys
from datetime import datetime

import pytest
from pyspark.sql import SparkSession

from relatorio_pedidos.io_utils.schemas import PagamentosSchema, PedidosSchema

# Garante que os workers Python do Spark usem o mesmo interpretador dos testes.
os.environ.setdefault("PYSPARK_PYTHON", sys.executable)
```

tests/conftest.py - linhas 14 a 32

```python
@pytest.fixture
def spark():
    """SparkSession local compartilhada; recriada se algum teste (ex.: Application) a encerrar."""
    sessao = (
        SparkSession.builder.appName("testes-relatorio-pedidos")
        .master("local[1]")
        .config("spark.sql.shuffle.partitions", "1")
        .config("spark.ui.enabled", "false")
        .config("spark.ui.showConsoleProgress", "false")
        .getOrCreate()
    )
    sessao.sparkContext.setLogLevel("ERROR")
    return sessao


def pytest_sessionfinish(session, exitstatus):
    ativa = SparkSession.getActiveSession()
    if ativa is not None:
        ativa.stop()
```

tests/conftest.py - linhas 35 a 51

```python
@pytest.fixture
def make_pedidos(spark):
    """Cria DataFrame de pedidos com o schema explícito do projeto.

    Cada linha: (id_pedido, valor_unitario, quantidade, data_criacao, uf).
    """

    def _make(linhas):
        dados = [(i, "PRODUTO", float(v), int(q), d, uf, 1) for i, v, q, d, uf in linhas]
        return spark.createDataFrame(dados, schema=PedidosSchema.get())

    return _make


@pytest.fixture
def make_pagamentos(spark):
    """Cria DataFrame de pagamentos com o schema explícito do projeto.
```

tests/conftest.py - linhas 53 a 63

```
    Cada linha: (id_pedido, forma_pagamento, status, fraude).
    """

    def _make(linhas):
        dados = [
            (i, forma, 100.0, status, datetime(2025, 1, 1), (fraude, 0.5))
            for i, forma, status, fraude in linhas
        ]
        return spark.createDataFrame(dados, schema=PagamentosSchema.get())

    return _make
```

## Testes unitários da lógica de negócio

tests/test_relatorio_pedidos.py - linhas 1 a 12

```python
import logging
from datetime import datetime

import pytest
from pyspark.sql.types import DoubleType, StringType, StructField, StructType, TimestampType

from relatorio_pedidos.business.relatorio_pedidos import RelatorioPedidos, RelatorioPedidosError


@pytest.fixture
def relatorio():
    return RelatorioPedidos(ano=2025)
```

tests/test_relatorio_pedidos.py - linhas 15 a 29

```python
def test_filtra_apenas_pagamentos_recusados_e_legitimos(relatorio, make_pagamentos):
    pagamentos = make_pagamentos(
        [
            ("recusado-legitimo", "PIX", False, False),
            ("recusado-fraude", "PIX", False, True),
            ("aprovado-legitimo", "PIX", True, False),
            ("aprovado-fraude", "PIX", True, True),
            ("status-nulo", "PIX", None, False),
            ("fraude-nula", "PIX", False, None),
        ]
    )

    ids = [r.id_pedido for r in relatorio.filtrar_pagamentos_recusados_legitimos(pagamentos).collect()]

    assert ids == ["recusado-legitimo"]
```

tests/test_relatorio_pedidos.py - linhas 32 a 44

```python
def test_filtra_apenas_pedidos_de_2025(relatorio, make_pedidos):
    pedidos = make_pedidos(
        [
            ("fim-2024", 10, 1, datetime(2024, 12, 31, 23, 59, 59), "SP"),
            ("inicio-2025", 10, 1, datetime(2025, 1, 1, 0, 0, 0), "SP"),
            ("fim-2025", 10, 1, datetime(2025, 12, 31, 23, 59, 59), "SP"),
            ("inicio-2026", 10, 1, datetime(2026, 1, 1, 0, 0, 0), "SP"),
        ]
    )

    ids = sorted(r.id_pedido for r in relatorio.filtrar_ano(pedidos).collect())

    assert ids == ["fim-2025", "inicio-2025"]
```

tests/test_relatorio_pedidos.py - linhas 47 a 65

```python
def test_ano_do_filtro_vem_da_injecao(make_pedidos):
    pedidos = make_pedidos(
        [
            ("a", 10, 1, datetime(2024, 5, 1), "SP"),
            ("b", 10, 1, datetime(2025, 5, 1), "SP"),
        ]
    )

    ids = [r.id_pedido for r in RelatorioPedidos(ano=2024).filtrar_ano(pedidos).collect()]

    assert ids == ["a"]


def test_calcula_valor_total_como_valor_unitario_vezes_quantidade(relatorio, make_pedidos):
    pedidos = make_pedidos([("p1", 150.5, 3, datetime(2025, 3, 1), "RJ")])

    linha = relatorio.calcular_valor_total(pedidos).collect()[0]

    assert linha.valor_total == pytest.approx(451.5)
```

tests/test_relatorio_pedidos.py - linhas 68 a 84

```python
def test_join_descarta_pedidos_sem_pagamento_filtrado(relatorio, make_pedidos, make_pagamentos):
    pedidos = make_pedidos(
        [
            ("p1", 10, 1, datetime(2025, 3, 1), "SP"),
            ("p2", 10, 1, datetime(2025, 3, 2), "SP"),
        ]
    )
    pagamentos = make_pagamentos([("p1", "PIX", False, False)])

    ids = [r.id_pedido for r in relatorio.juntar(pedidos, pagamentos).collect()]

    assert ids == ["p1"]


def test_colunas_finais_do_relatorio(relatorio, make_pedidos, make_pagamentos):
    pedidos = relatorio.calcular_valor_total(make_pedidos([("p1", 10, 2, datetime(2025, 3, 1), "SP")]))
    pagamentos = make_pagamentos([("p1", "BOLETO", False, False)])
```

tests/test_relatorio_pedidos.py - linhas 86 a 105

```
86      df = relatorio.selecionar_colunas(relatorio.juntar(pedidos, pagamentos))
87
88      assert df.columns == ["id_pedido", "uf", "forma_pagamento", "valor_total", "data_pedido"]
89      assert df.columns == RelatorioPedidos.COLUNAS_RELATORIO
90
91
92  def test_ordena_por_uf_forma_pagamento_e_data(relatorio, spark):
93      schema = StructType(
94          [
95              StructField("id_pedido", StringType()),
96              StructField("uf", StringType()),
97              StructField("forma_pagamento", StringType()),
98              StructField("valor_total", DoubleType()),
99              StructField("data_pedido", TimestampType()),
100         ]
101     )
102     df = spark.createDataFrame(
103         [
104             ("d", "SP", "PIX", 1.0, datetime(2025, 1, 1)),
105             ("c", "SP", "BOLETO", 1.0, datetime(2025, 2, 1)),
```

tests/test_relatorio_pedidos.py - linhas 106 a 114

```
106             ("b", "AC", "PIX", 1.0, datetime(2025, 3, 1)),
107             ("a", "AC", "PIX", 1.0, datetime(2025, 1, 15)),
108         ],
109         schema=schema,
110     )
111
112     ids = [r.id_pedido for r in relatorio.ordenar(df).collect()]
113
114     assert ids == ["a", "b", "c", "d"]
```

tests/test_relatorio_pedidos.py - linhas 117 a 135

```python
def test_gerar_relatorio_de_ponta_a_ponta(relatorio, make_pedidos, make_pagamentos):
    pedidos = make_pedidos(
        [
            ("ok-sp", 100, 2, datetime(2025, 6, 1, 10), "SP"),
            ("ok-ac", 50, 3, datetime(2025, 7, 1, 10), "AC"),
            ("aprovado", 100, 1, datetime(2025, 6, 1), "SP"),
            ("fraude", 100, 1, datetime(2025, 6, 1), "SP"),
            ("ano-2024", 100, 1, datetime(2024, 6, 1), "SP"),
        ]
    )
    pagamentos = make_pagamentos(
        [
            ("ok-sp", "PIX", False, False),
            ("ok-ac", "BOLETO", False, False),
            ("aprovado", "PIX", True, False),
            ("fraude", "PIX", False, True),
            ("ano-2024", "PIX", False, False),
        ]
    )
```

tests/test_relatorio_pedidos.py - linhas 137 a 154

```
    linhas = relatorio.gerar_relatorio(pedidos, pagamentos).collect()

    assert [(r.id_pedido, r.uf, r.forma_pagamento, r.valor_total) for r in linhas] == [
        ("ok-ac", "AC", "BOLETO", 150.0),
        ("ok-sp", "SP", "PIX", 200.0),
    ]
    assert linhas[0].data_pedido == datetime(2025, 7, 1, 10)


def test_erro_em_etapa_e_logado_e_relancado_de_forma_controlada(relatorio, make_pagamentos, spark, caplog):
    pedidos_sem_data = spark.createDataFrame(
        [("p1", "SP")], StructType([StructField("id_pedido", StringType()), StructField("uf", StringType())])
    )
    pagamentos = make_pagamentos([("p1", "PIX", False, False)])

    with caplog.at_level(logging.ERROR):
        with pytest.raises(RelatorioPedidosError, match="Filtro do ano"):
            relatorio.gerar_relatorio(pedidos_sem_data, pagamentos)
```

tests/test_relatorio_pedidos.py - linhas 156 a 168

```
    assert any("Falha na etapa 'Filtro do ano'" in m for m in caplog.messages)


def test_registra_contagem_de_cada_etapa(relatorio, make_pedidos, make_pagamentos, caplog):
    pedidos = make_pedidos([("p1", 10, 1, datetime(2025, 3, 1), "SP")])
    pagamentos = make_pagamentos([("p1", "PIX", False, False)])

    with caplog.at_level(logging.INFO):
        relatorio.gerar_relatorio(pedidos, pagamentos)

    etapas = [m for m in caplog.messages if m.startswith("Etapa '")]
    assert len(etapas) == 6
    assert all("1 registros" in m for m in etapas)
```

## Testes da orquestração com dependências injetadas (mocks)

tests/test_pipeline.py - linhas 1 a 16

```python
from datetime import datetime
from unittest.mock import MagicMock

import pytest

from relatorio_pedidos.business.relatorio_pedidos import RelatorioPedidos, RelatorioPedidosError
from relatorio_pedidos.pipeline.pipeline import Pipeline


def test_pipeline_orquestra_leitura_transformacao_e_escrita(make_pedidos, make_pagamentos):
    reader, writer = MagicMock(), MagicMock()
    reader.read_pedidos.return_value = make_pedidos([("p1", 10, 1, datetime(2025, 3, 1), "SP")])
    reader.read_pagamentos.return_value = make_pagamentos([("p1", "PIX", False, False)])
    writer.write.return_value = "saida"

    caminho = Pipeline(reader, RelatorioPedidos(ano=2025), writer).run()
```

tests/test_pipeline.py - linhas 18 a 32

```python
    assert caminho == "saida"
    reader.read_pedidos.assert_called_once()
    reader.read_pagamentos.assert_called_once()
    df_gravado = writer.write.call_args.args[0]
    assert [r.id_pedido for r in df_gravado.collect()] == ["p1"]


def test_pipeline_nao_grava_quando_a_regra_de_negocio_falha():
    reader, writer, relatorio = MagicMock(), MagicMock(), MagicMock()
    relatorio.gerar_relatorio.side_effect = RelatorioPedidosError("falhou")

    with pytest.raises(RelatorioPedidosError):
        Pipeline(reader, relatorio, writer).run()

    writer.write.assert_not_called()
```

## Execução do pipeline (logs)

python main.py - linhas 1 a 10

```
2026-10-13 21:37:23,023 - INFO - Pipeline iniciado
2026-10-13 21:37:23,708 - INFO - Iniciando a geração do relatório (ano 2025)
2026-10-13 21:37:25,850 - INFO - Etapa 'Filtro do ano' concluída: 31680 registros
2026-10-13 21:37:26,231 - INFO - Etapa 'Cálculo do valor total' concluída: 31680 registros
2026-10-13 21:37:27,528 - INFO - Etapa 'Filtro de pagamentos recusados e legítimos' concluída: 2747 registros
2026-10-13 21:37:28,107 - INFO - Etapa 'Join pedidos x pagamentos' concluída: 1473 registros
2026-10-13 21:37:28,370 - INFO - Etapa 'Seleção de colunas' concluída: 1473 registros
2026-10-13 21:37:28,632 - INFO - Etapa 'Ordenação' concluída: 1473 registros
2026-10-13 21:37:28,632 - INFO - Relatório gerado com sucesso
2026-10-13 21:37:31,884 - INFO - Pipeline concluído. Parquet gravado em: data/output/relatorio_pedidos
```

## Validações do parquet

python scripts/validar_saida.py - linhas 1 a 8

```
[OK] colunas == ['id_pedido', 'uf', 'forma_pagamento', 'valor_total', 'data_pedido']
[OK] contagem == 1473 (cálculo independente)
[OK] id_pedido sem duplicidade
[OK] todos os pedidos de 2025
[OK] todos com status=false e fraude=false
[OK] valor_total == valor_unitario * quantidade
[OK] ordenado por uf, forma_pagamento, data_pedido
Total de linhas: 1473
```

## Amostra do parquet (schema e primeiras linhas)

python scripts/validar_saida.py - linhas 1 a 19

```
root
 |-- id_pedido: string (nullable = true)
 |-- uf: string (nullable = true)
 |-- forma_pagamento: string (nullable = true)
 |-- valor_total: double (nullable = true)
 |-- data_pedido: timestamp (nullable = true)
+------------------------------------+---+---------------+-----------+-------------------+
|id_pedido                           |uf |forma_pagamento|valor_total|data_pedido        |
+------------------------------------+---+---------------+-----------+-------------------+
|acbf9b42-eaca-4cd9-914a-109de59aa943|AL |BOLETO         |2500.0     |2025-02-02 17:02:14|
|df280f1a-b8f4-4a29-be64-66fb7fab2a25|AL |BOLETO         |1100.0     |2025-06-12 12:29:22|
|394c5640-fae2-413f-bdfb-52f316fa6782|AL |CARTAO_CREDITO |5000.0     |2025-01-03 18:49:13|
|4440113d-717d-4887-8889-574853e47e20|AL |CARTAO_CREDITO |1100.0     |2025-03-06 06:33:11|
|cdbf1791-ad2f-4357-af93-1af7e9545214|AL |CARTAO_CREDITO |600.0      |2025-03-15 13:24:44|
|a66abc7f-d362-4942-8be9-e6f7b696b14e|AL |CARTAO_CREDITO |300.0      |2025-03-27 20:28:37|
|9b459f36-d5b9-4b1a-91d9-0be858e369dc|AL |CARTAO_CREDITO |30.0       |2025-04-18 16:21:57|
|f70efb0f-6457-4356-aa20-a72c7fb0b45f|AL |CARTAO_CREDITO |1500.0     |2025-04-27 11:16:35|
+------------------------------------+---+---------------+-----------+-------------------+
only showing top 8 rows
```

## Execução dos testes (pytest)

pytest -v - linhas 1 a 11

```
============================= test session starts ==============================
collecting ... collected 19 items
tests/test_integracao.py::test_schemas_sao_explicitos_e_incluem_o_objeto_aninhado PASSED [  5%]
tests/test_integracao.py::test_escrita_em_parquet_preserva_os_dados PASSED [ 10%]
tests/test_integracao.py::test_aplicacao_executa_o_pipeline_completo PASSED [ 15%]
tests/test_integracao.py::test_aplicacao_retorna_erro_quando_a_entrada_nao_existe PASSED [ 21%]
tests/test_pipeline.py::test_pipeline_orquestra_leitura_transformacao_e_escrita PASSED [ 26%]
tests/test_pipeline.py::test_pipeline_nao_grava_quando_a_regra_de_negocio_falha PASSED [ 31%]
tests/test_relatorio_pedidos.py::test_filtra_apenas_pagamentos_recusados_e_legitimos PASSED [ 36%]
tests/test_relatorio_pedidos.py::test_filtra_apenas_pedidos_de_2025 PASSED [ 42%]
tests/test_relatorio_pedidos.py::test_ano_do_filtro_vem_da_injecao PASSED [ 47%]
```

pytest -v - linhas 12 a 22

```
tests/test_relatorio_pedidos.py::test_calcula_valor_total_como_valor_unitario_vezes_quantidade PASSED [ 52%]
tests/test_relatorio_pedidos.py::test_join_descarta_pedidos_sem_pagamento_filtrado PASSED [ 57%]
tests/test_relatorio_pedidos.py::test_colunas_finais_do_relatorio PASSED [ 63%]
tests/test_relatorio_pedidos.py::test_ordena_por_uf_forma_pagamento_e_data PASSED [ 68%]
tests/test_relatorio_pedidos.py::test_gerar_relatorio_de_ponta_a_ponta PASSED [ 73%]
tests/test_relatorio_pedidos.py::test_erro_em_etapa_e_logado_e_relancado_de_forma_controlada PASSED [ 78%]
tests/test_relatorio_pedidos.py::test_registra_contagem_de_cada_etapa PASSED [ 84%]
tests/test_settings.py::test_padroes_quando_nao_ha_yaml PASSED           [ 89%]
tests/test_settings.py::test_yaml_sobrescreve_padroes PASSED             [ 94%]
tests/test_settings.py::test_variaveis_de_ambiente_tem_precedencia_sobre_o_yaml PASSED [100%]
============================== 19 passed in 55.77s ==============================
```